

Product OS Leve — Mandor

Baseado em "O Manual do Fundador: Estágio do Lançamento" (p. 44-45)

****Versão:**** 0.1 (rascunho)

****Data:**** 2026-06-02

****Objetivo:**** Processos leves e repetíveis que rodem sem intervenção constante do fundador

1. CADÊNCIA DE SPRINT

Ritmo

- ****Sprint:**** 2 semanas (10 dias úteis)
- ****Cerimônias:****
 - Seg 9:30 — Sprint Planning (1h, segunda-feira da semana de sprint)
 - Qua 15:00 — Sprint Sync (30min, mid-sprint check)
 - Sex 17:00 — Sprint Review + Retro (1.5h, sexta-feira)
- ****Participantes:**** Fundador + tech lead + product (se houver)

Template de Sprint

SPRINT [N] — [Datas]

Goal: [1 frase: o que queremos alcançar]

FEATURES (o que estamos construindo)

- Feature A (estimado: 3 dias)
- Feature B (estimado: 2 dias)
- Feature C (estimado: 2 dias)

OPERACIONAL (manutenção, bugs priority-1)

- Critical bug X (estimado: 1 dia)
- Refactor Y (estimado: 1 dia)

METRICS (o que iremos medir)

- Deployment frequency: target 2x/semana
- Critical bugs: target <3 open
- Test coverage: target >75%

Regras de Escopo

- ****1 sprint = 1 goal****
- Não carregar features que não caibam em 10 dias
- Deixar 1 dia flutuante para P0 bugs ou urgências
- Refactor/tech debt = máx 20% do sprint

2. TEMPLATE MÍNIMO DE SPEC

****Quando:**** Antes de qualquer feature iniciar codificação

****Quem rascunha:**** Fundador ou PM; revisado por tech lead

****Onde:**** GitHub issue + markdown file em `/specs/`

```markdown

**Spec: [Feature Name]**

## Problema

[1–2 parágrafos: qual é o problema real do usuário?]

Exemplo:

"Analistas gastam 30min cada um em validação manual de integrity de dados"  
"(chegar se vendas batem com custos, se balanço fecha). Erros passam."  
"Queremos automatizar."

## Solução Proposta

[1–2 parágrafos: o que estamos construindo]

Exemplo:

"Rodar integrity check automático antes de relatório final."  
"Alertar se inconsistências. Bloquear release do relatório até resolver."

## Escopo: Faz vs. Não Faz

### Faz (MVP desta feature)

- Detecta 5 tipos de inconsistência (lista em APPENDIX A)
- Bloqueia relatório se erros críticos
- Mostra error message clara ao analista
- API endpoint retorna `integrity\_check\_result` em JSON

### Deliberadamente NÃO faz (Fase 2)

- ~~Auto-fix inconsistências~~ (manual review necessário)
- ~~Alertas em Slack~~ (apenas no UI por enquanto)
- ~~Histórico de audits~~ (salvo em logs, não em DB)

## Critérios de Aceição

- Testes cobrem 5 inconsistências da APPENDIX A
- Relatório bloqueado + clear error message quando `is\_blocked: true`
- API retorna em <200ms mesmo com dataset grande (stress test)
- Documentado em /docs/integrity-check.md

## Estimativa

- Codificação: 3 dias (dev) + 1 dia (testes) + 0.5 dia (code review)
- Total: 1 sprint

## Dependências

- APPENDIX A finalizado (especificar 5 regras exatas)
- Database migration (nova coluna integrity\_status) — em sprint anterior

## APPENDIX A: Regras de Integridade

1. **\*\*Vendas + Custos = Total\*\*** —  $\text{sum}(\text{vendas}) + \text{sum}(\text{custos}) = \text{total\_line\_item}$  (tolerância:  $\pm 0.5\%$ )
2. **\*\*Balanço fecha\*\*** —  $\text{Assets} = \text{Liabilities} + \text{Equity}$  (tolerância:  $\pm 0.1\%$ )
3. [...]

---

---

---

### 3. ÁRVORE DE DECISÃO: TRIAGEM DE BUGS

**\*\*Quando entra:\*\*** Relatório de bug via GitHub issue / Slack / usuário

**\*\*Quem faz:\*\*** Tech lead (15 min SLA)

**\*\*Output:\*\*** Label + milestone + assigned dev

---

BUG ENTRA

↓

[Consegue reproduzir?]

■ ■ NÃO → Label "needs-info" + pedir mais detalhe ao reporter

■ ■ SIM → continua...

↓

[Afeta produção / dados de cliente?]

■ ■ SIM → Priority P0 (bloqueia tudo)

■ • Assign imediatamente

■ • SLA: corrigir em 24h

■ ■ NÃO → continua...

↓

[Afeta feature crítica (análise, reports)?]

■ ■ SIM → Priority P1 (próximo sprint)

■ • Assign para próximo ciclo de trabalho

■ • SLA: corrigir em 1 sprint

■ ■ NÃO → continua...

↓

[UX/performance/doc issue?]

■ ■ SIM → Priority P2 (nice-to-have)

■ • Label "p2-ux" ou "p2-perf"

■ • Backlog, considerar em sprints futuros

■ ■ NÃO → Priority P3 (muito menor)

• Fecha se não há impacto real

REGRA OURO: P0 bugs interrompem sprint. P1+ esperam próximo sprint.

---

#### SLAs de Bug

| Severidade | SLA Resposta | SLA Fix | Exemplo |

|-----|-----|-----|-----|

| P0 | 2h | 24h | Análise trava, perdem dados |

| P1 | 24h | 1 sprint | Relatório lento, layout quebrado |

| P2 | 1 semana | backlog | Typo, minor UX tweak |

| P3 | N/A | N/A | Documentação, nice-to-have |

---

### 4. BRIEFING SEMANAL DE MÉTRICAS

**\*\*Quando:\*\*** Toda sexta-feira, 17:00 (após Sprint Review)

**\*\*Quem rascunha:\*\*** Engenheiro ou analytics + Claude Cowork

**\*\*Formato:\*\*** 1 página markdown, salvo em `/metrics/week-[N].md`

#### Template

```markdown

Weekly Metrics — Semana [N] ([Data])

■ Sprint Goal (estávamos tentando quê?)

[1 frase do sprint anterior]

■ Key Metrics (vs. target)

Product

- **Analyses complete:** 47 (target: 40) ■ +17%
- **Avg analysis time:** 6.2h (target: <8h) ■
- **Error rate (P0 bugs):** 0 (target: 0) ■
- **Test coverage:** 73% (target: >75%) ■■ (trend: up 2%)

Platform

- **Uptime:** 99.7% (target: 99.5%) ■
- **API p99 latency:** 180ms (target: <200ms) ■
- **Deploy frequency:** 2x/week ■
- **Critical bugs open:** 1 (target: 0) ■■

Business

- **New customers:** 2 (target: 2) ■
- **ARR added:** \$120K (target: \$100K) ■
- **Churn:** 0 (target: 0) ■
- **Net retention:** 125% (target: >120%) ■

■ Issues & Blockers

| Issue | Severity | Owner | Status |
|-------------------------|----------|--------|----------------------------|
| ----- ----- ----- ----- | | | |
| Test coverage falling | Low | [Name] | On track to fix sprint N+1 |
| [Bug X] P1 escalated | Medium | [Name] | Fix in progress |

■ Trend Analysis

- **Good:** Análises aceleradas, ARR subiu mais que expected.
- **Watch:** Test coverage trending down 2 sprints; refactor scheduled S+1.
- **Decision needed:** Customer X quer feature nova; descartar ou adicionar roadmap?

■ Action Items

- Refactor test suite (sprint N+1) — @tech-lead
 - Customer X call (decidir feature) — @founder
 - Blog post "Mandor 2026 track record" — @marketing
-
- ...

Métricas Incluídas

- **Product:** Qualidade, performance, uso do cliente
- **Platform:** Infraestrutura, uptime, deploy
- **Business:** Growth, retention, health

Distribuição

- Slack: resumo (3 parágrafos) + link para PDF
 - Email semanal: para investidores/board (se aplica)
 - Interno: arquivo completo em Git
-

5. REGRAS DE DEPLOYMENT

Pré-Deploy Checklist

- Código passou em CI/CD (testes verdes, no security warnings)
- Code review aprovado por tech lead
- Specs/migrations documentadas
- Rollback plan escrito (como revert se quebrar?)
- Pessoa de on-call notificada (quem monitora pós-deploy?)

Deploy Cadência

- **Produção:** 2x por semana (terça e quinta, 14:00 UTC)
- **Staging:** 1x por dia (acompanhar development)
- **Hotfix:** Qualquer hora (P0 bugs)

Pós-Deploy

- **Verificação:** 10 minutos após deploy, rodar smoke tests
- **Monitoramento:** 1h de observação ativa (logs, metrics, user reports)
- **Rollback:** Prompt se detectado P0 issue; revert em <5min

6. CONTROLE OPERACIONAL (Claude Cowork)

O que Claude Cowork roda recorrentemente:

Diário

- Triagem de bugs chegados no dia anterior
- Escalação de P0s (Slack alert ao on-call)
- Atualizar sprint board com avanço

Semanal (terça)

- Rascunhar briefing de métricas
- Enviar relatório de segurança/uptime
- Preparar agenda de Sprint Planning

Semanal (sexta)

- Compilar métricas finais
- Enviar briefing para stakeholders
- Limpar backlog (remove duplicatas, resolve "needs-info")

Mensal (último sexta)

- Revisar roadmap
- Capturar feedback de usuários (síntese)
- Identificar mudanças de prioridade

7. REGRAS DE PRODUTO

Princípios (não negociáveis)

1. ****Funcionalidade > Beleza****

MVP resolve problema real. Design é iteração.

2. ****Segurança > Velocidade****

Security review antes de feature tocar produção.

3. ****Dados > Intuição****

Decisões de produto baseadas em métricas + feedback real, não em "sounds cool".

4. ****Escopo Escrito > Feature Creep****

Cada feature tem spec. Emendas exigem evidência de usuários.

5. ****Teste > Esperança****

Código sem testes não vai para produção. (exceção: hotfix P0, depois escrever testes).

Critérios para Aceitar Nova Feature (no backlog)

■ Problema real de usuário? (qual usuário, quanto tempo economiza?)

■ Pode ser feito em 1 sprint? (>10 dias = split ou defer)

■ Spec escrito? (Problema + Solução + Faz/Não faz)

■ Alinhado com roadmap de 6 meses?

Se não atende todos: ****deferido****. Backlog é priorizado, não infinito.

8. EXEMPLO: Sprint N em Ação

****Data:**** 2026-06-09 a 2026-06-20

****Goal:**** "Integridade de dados automática (feature X)"

Sprint Planning (Seg 09:06)

- Revisam spec (já escrito no backlog)
- Estimam: 3 dev + 1 qa + 0.5 review = 4.5 dias
- Cabe? Sim (8 dias disponíveis)
- Assinar: @dev1, @qa1, @lead

Meio-Sprint Sync (Qua 15:06)

- Dev1: "Implementação 70%, começando testes amanhã"
- QA1: "Aguardando build em staging"
- Lead: "On track"

Sprint Review + Retro (Sex 17:06)

- Demo feature: "Aqui carrego dados, rodam integrity checks, bloqueia se erro"
- Testes: 100% de cobertura dos 5 casos
- Métrica: API <150ms, uptime 99.9%
- Retro: "Spec muito claro, dev foi smooth. Próxima vez pedir staging mais cedo."

Deploy (Ter 14:06, próxima semana)

- Rodar smoke tests ■
- Monitor 1h ■
- Usuários começam a usar ■

9. PRÓXIMAS AÇÕES

- Agendar Sprint Planning (próxima segunda)
- Escrever 3 specs (features backlog mais urgentes)
- Setup: GitHub labels (P0, P1, P2, P3, needs-info)
- Designar tech lead como SPOC de triagem de bugs
- Configurar alertas de P0 no Slack

